

Software Testing Report

Cohort 3

Group 11

Team Aubergine

Joshua Wainwright, Piotr Koziol, Sarvesh Sridhar, Harrison
Barrans, Daniel Thwaites, Callum Newton, Arnav Jamidar,
Harry Turner

Part A - Testing Methods and Approaches

For the software testing we decided that it would be best to implement white box testing, which ensures that all requirements are met and working which is what is needed for a project like this.

For this project we are not aiming for 100% coverage because we are only creating a small game whereas if we were designing a medical device then 100% coverage would certainly be needed. Users will be encouraged to report any bugs that they find to the developers which can then be fixed at a later date. By doing this we can ensure that we hit our deadline and no major bugs will be in the game.

Automated Testing

Our test plan focuses mainly on automated unit tests. These check the correctness of each section of our code independently from other sections, which helps to ensure that bugs are not introduced as the code is developed. These tests cover the internal code of the game, focusing on high level game mechanics rather than arbitrary implementation details.

Furthermore, our unit tests require the entire game to be compiled and loaded, before classes relevant to the test are used. This behaviour is a necessity due to our build system, Gradle. However, it also acts as a smoke test which catches syntax errors and missing files even in areas of the code that are not otherwise automatically tested.

All of our automated tests were written by team members working on the implementation, since they were already familiar with the codebase. The tests were executed by our continuous integration infrastructure after every commit, and results reported back to the team via GitHub's user interface. The results were analysed by those responsible for this testing report.

Manual Testing

Finally, manual tests were used to evaluate the usability and visual appearance of the game, since these are qualitative factors. These tests cover the user interface which is presented, and are meant to be executed without knowledge of the underlying implementation.

Manual test plans were developed by those responsible for this testing report, then carried out on the final version of the game. For the reduction of workload, we did not deem it necessary to manually test the game after every iteration, however developers were encouraged to carry out ad-hoc tests after each change they made.

Manual tests will be carried out on our development machines. This is because we have already ensured that they are set up with the correct environment (operating system and Java version) to meet the client's requirements, so we can be sure that the game functions under those conditions.

Part B - Explanation of Test Results

Automated Testing

A total of 22 automated tests were run using JUnit, all of which passed successfully. Code coverage analysis using JaCoCo reported 34% line coverage and 13% branch coverage.

Test No	Test	Status	Link to requirement
1	Check that entities with various behaviours can be created and removed	Pass	FR-13
2	Check if the leaderboard shows correctly and updates when new scores are added	Pass	UR-15
3	Check if text bubbles appear and disappear when they are meant to	Pass	FR-05
4	Ensures that only one bob can be present at all time	Pass	NFR-08
5	Checks that items are picked up when bob is over them.	Pass	FR-05
6	Checks if bob is currently colliding with an object and stops the players movement	Pass	FR-05
7	Checks that the game window can be resized to fit the users need	Pass	FR-12
8	Checks that the correct screen shows to the player in the correct scenario (win, lose, tutorial, title)	Pass	FR-03

Manual Testing

Test no	Expected Outcome	Actual Outcome	Status
9	Game opens and runs smoothly on different devices	Game opened and ran smoothly on different devices	Pass
10	Timer is visible on the screen	Timer is visible on the screen	Pass
11	The positive, negative or hidden events counter will increase after interactions. Text bubbles will also appear for certain interactions	The positive, negative or hidden event counter increased based on the event. Text bubbles appeared for certain events	Pass
12	Game pauses after pressing ESC	The Game paused and the pause	Pass

	and the pause screen pops up	screen appeared after pressing the ESC key	
13	Tutorial screen has explanations for the controls and also lists the goals/objectives on the side	Tutorial screen had explanations for the controls and also outlined the main goals of the game	Pass
14	The art and UI theme stays consistent throughout the whole playthrough	The art and UI theme stayed consistent throughout the whole playthrough	Pass
15	Text message appears to notify the user once they have attained an achievement ('Completionist' is displayed after interacting with all the events in the game)	Text message ('Completionist') appeared on screen after the achievement was earned	Pass
16	Leaderboard appears with the highest recent scores at the end of the game	Leaderboard appeared at the end of the game with the recent highest scores	Pass
17	The User is able to hear background music whilst running the game. The user also sees/hears SFX when interacting with events	After running the game no background noises or SFX were heard or seen	Fail
18	Certain events only appear on some runs but not others	Events always appear in the same locations and are unconditionally triggered	Fail
19	No player data is stored/displayed from previous runs	Player's past high scores were displayed on the Leaderboard at the end of the game	Fail

Most of our manual tests passed, except for the following: Test 17 failed as we do not have any background music or SFX in the game. Test 18 failed as we have not implemented events that have a random chance of occurring during the game. Test 19 failed as we have stored player data to display the top scores on the leaderboard at the end of every run.

In order for these tests to pass, we would have to add background music and sound effects to the game - these assets would need to be created. We would also need to adjust some of the existing events so that they are presented in randomised locations, or only appear in a certain percentage of runs.

Req id	Reqs tested	FR-1	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9	FR-10	FR-11	FR-12	FR-13	UR-15	UR-16	NFR-8
Test cases	17	1	1	1	4	1	1	1	1	1	1	1	1	2	1	1
1	1												x			
2	1													x		
3	1				x											
4	1															x
5	1				x											
6	1				x											
7	1											x				
8	1		x													
9	1	x														
10	1			x												
11	1				x											
12	1					x										
13	1						x									
14	1							x								
15	1														x	
16	1													x		
17	1								x							
18	1									x						
19	1										x					

Part C - Evidence of Tests

JUnit test reports and JaCoCo coverage reports for the final version of our game are available at <https://uoy-eng1-3-11.github.io/website-2/#testMenu>. The following table describes the manual test cases we used.

Test no	Description	Related Req	Steps to be followed
9	Verify that the game launches and runs on standard desktop operating systems using Java	FR-01	Run the game on multiple devices and check whether it starts successfully without errors
10	Verify that the game shows a 5 minute timer to the user	FR-04	Run the game and identify that the 5 minute timer appears
11	Verify that the game provides visual or textual feedback to the user after game interactions	FR-05	Run the game and interact with all obstacles and observe the feedback given to the user by the game
12	Verify that the game provides an option for the user to pause/resume	FR-06	Run the game and press ESC for the pause screen to pop up
13	Verify that the tutorial screen explains controls and objectives	FR-07	Run the game and go into the tutorial to identify explanations for controls and objectives
14	Verify that the game provides a consistent art and UI theme	FR-08	Run the game and notice the art and UI theme throughout the playthrough of the game
15	Verify that the game has achievements upon completing certain tasks in-game	UR-16	Run the game and identify if any achievements are attained after completion of certain tasks in the game
16	Verify that the game has a leaderboard at the end of the playthrough with the top scores displayed of the recent runs	UR-15	Run the game and complete a playthrough. Observe if a leaderboard appears at the end with the scores of the top recent runs displayed in order
17	Verify that the game has background music or SFX for events	FR-09	Run the game and listen for background music or SFX for events throughout the playthrough
18	Verify that the game randomly spawns hidden or humorous events at times	FR-10	Run the game multiple times and identify if certain events appear during certain runs but not others
19	Verify that the game does not store any player data	FR-11	Run the game and notice if the game displayed any data from previous runs during the current run

